

Short answer

Yes, this is a strong BSc thesis topic. However, “I ported PAQ7/PAQ8 to CUDA” is no longer enough by itself for a significant paper.

A highly relevant paper appeared at **Data Compression Conference 2026**:

CuCM: A GPU-Powered Context-Mixing Compressor for Archival Storage

CuCM addresses the same fundamental challenge using **pre-learning, batched model updates, vector-level prediction, and speculative look-ahead decompression**. It reports up to **12.6× higher throughput than LPAQ with comparable compression ratios** [79]. Consequently, you must position your work against CuCM—not claim that GPU context mixing itself is new.

Assessment of your current material

Your `/workspace/paq7.cpp` is essentially an old, Windows-oriented PAQ7 implementation:

- It performs adaptive prediction and learning **one bit at a time**.
- The arithmetic encoder depends on each immediately preceding prediction.
- Context maps are updated after every bit.
- The mixer’s neural weights are trained after every bit.
- It uses global mutable state extensively.
- It contains DLL/KGB-archiver integration and legacy Microsoft-specific code.
- It is not currently a CUDA implementation.

These dependencies are exactly what make a direct kernel conversion ineffective. One CUDA thread processing every bit would probably be **slower than an optimized CPU**, while assigning bits to independent threads would change the model state and compressed output.

The PDF correctly recognizes this sequential learning dependency, but its related-work section needs a major update: **CuCM is now the closest and most important prior work**.

Where genuine novelty could come from

1. PAQ-specific batched or delayed learning

Develop a mathematically defined approximation in which a CUDA warp predicts several bits before applying model updates.

Study:

- batch/vector length (V),
- compression-ratio loss,
- throughput gain,
- divergence and memory behavior,
- delayed versus accumulated gradient updates.

This is more defensible than merely placing PAQ functions inside CUDA kernels. However, it overlaps strongly with CuCM, so your mechanism must differ or improve on it.

2. Warp-parallel PAQ mixer and model ensemble

PAQ7 evaluates many model predictions and combines them through mixer dot products. You could map:

- one lane or thread to each prediction model,
- warp reductions to the mixer’s weighted sum,
- vector instructions or tensor-oriented operations to multiple mixers,
- cooperative groups to model evaluation.

The novelty would be a **dependency-preserving mapping of heterogeneous PAQ models**, not merely GPU arithmetic.

The danger is that PAQ7 has relatively small mixer operations, irregular hash-table accesses, and bit-level synchronization. Kernel-launch and memory costs can exceed the useful computation. A persistent kernel would probably be necessary.

3. Speculative multi-path prediction

For the next (k) bits, evaluate possible histories speculatively and retain the path selected by the arithmetic decoder.

For example:

- predict both branches for one future bit;
- evaluate (2^k) possible paths for small (k);
- prune or share common state;
- quantify the useful look-ahead limit.

CuCM already uses aggressive look-ahead during decompression [79], so novelty would require a **better state-sharing, pruning, or warp-level speculation method specifically for PAQ7/PAQ8**.

4. Block-parallel PAQ with state warm-up

Divide input into independently compressed blocks, but reduce ratio loss through:

- an overlap/warm-up prefix,
- transferred context summaries,
- sampled model state,
- shared global priors,
- file-type-aware boundaries.

ZPAQ already compresses independent blocks in multiple CPU threads [46], so plain block independence is not novel. A new contribution would be a method that gives a substantially better **speed/ratio trade-off** than naïve block splitting.

This is probably the **most realistic BSc implementation path**.

5. Batched compression of many independent files or streams

Run one PAQ state machine per CUDA thread block and process hundreds of small files concurrently. This does not parallelize one stream, but it offers natural GPU parallelism without changing PAQ semantics.

Potential contribution:

- exact PAQ-compatible concurrent compression;
- GPU-resident memory allocator and state layout;
- structure-of-arrays transformation;
- persistent work scheduling for unequal file sizes;
- throughput and energy evaluation.

This is publishable only if you target a credible workload such as backup archives, object storage, or genomic shards and demonstrate substantial aggregate throughput. It is less algorithmically novel but much safer technically.

6. PAQ8-oriented model redesign

PAQ8 is a better research target than PAQ7 if you can use a clean and legally distributable source version. Possible contribution:

- identify PAQ8 components responsible for most runtime;
- redesign only those components for GPU execution;
- retain the CPU for inherently sequential control;
- overlap GPU prediction with CPU coding;
- compare hybrid, GPU-only, and CPU-only variants.

This could become a good **heterogeneous-system paper**, especially if you derive a general method applicable to context-mixing compressors rather than only PAQ8.

My recommended thesis direction

Proposed title

GPU Acceleration of Adaptive Context-Mixing Compression: Dependency-Aware Parallelization of PAQ

Do not make “CUDA implementation of PAQ7” the thesis title. It sounds like an engineering port. Frame it as a study of **parallelizing autoregressive adaptive compressors**.

Central research question

How can PAQ’s bit-level prediction and online model adaptation be reorganized for GPU execution while minimizing degradation in compression ratio?

Recommended design

Implement and compare three variants:

1. **Exact baseline**
Clean, reproducible CPU PAQ7/PAQ8 implementation.
2. **Independent block-parallel CUDA PAQ**
Blocks processed independently, with configurable block sizes.
3. **Warm-start or delayed-update CUDA PAQ**
Your novel mechanism to recover compression ratio or increase within-stream parallelism.

That gives you a clear scientific experiment rather than only a software deliverable.

What would count as a significant achievement?

A credible paper should demonstrate at least one of these:

- **5×–10× end-to-end speedup** over an optimized single-core implementation with less than roughly **1–3% relative compressed-size degradation**;
- clear improvement over naïve block-parallel PAQ in the speed/ratio Pareto frontier;
- exact-format compatibility with substantial throughput on many concurrent streams;
- faster decompression through an original speculation/state-sharing mechanism;
- a method that generalizes across both PAQ7 and PAQ8;
- strong energy-efficiency gains, such as significantly better MB/J;
- an insightful negative result showing, through profiling and dependency analysis, precisely why different PAQ components do not benefit from GPUs—although this is harder to publish as a full paper.

“Kernel X is 20× faster” is insufficient if host transfers and sequential portions leave the complete compressor only 1.2× faster. Report **end-to-end compression and decompression throughput**.

Essential experiments

Baselines

Compare against:

- original PAQ7 and/or the exact relevant PAQ8 variant;
- optimized CPU version with compiler optimizations;
- multicore block-parallel implementation;
- LPAQ if reproducible;
- ZPAQ;
- modern practical compressors such as Zstd and LZMA/7-Zip;
- CuCM, if code becomes available, or at minimum compare methodology and published results.

Datasets

Use several established corpora and larger modern data:

- Calgary Corpus;
- Canterbury Corpus;
- Silesia Corpus;
- enwik8 and possibly enwik9;
- mixed binaries;
- source code;
- executables;
- images and already-compressed data;
- many-small-file archival workloads.

Do not draw conclusions from a single text file.

Metrics

Report:

- compressed size and bits per byte;
- compression and decompression MB/s;
- end-to-end speedup;
- GPU utilization and kernel occupancy;
- memory bandwidth and cache behavior;
- CPU/GPU energy if measurable;
- device memory consumption;
- initialization and transfer overhead;
- performance versus block/vector size.

Use repeated trials, confidence intervals, identical input ordering, and fixed hardware/software details.

Ablation studies

Individually disable:

- delayed updates;
- warm-up regions;

- speculative prediction;
- individual GPU-offloaded models;
- shared-memory caching;
- persistent kernels;
- different state layouts.

This is important for publication: it proves which idea caused the improvement.

PAQ7 or PAQ8?

My recommendation:

- Use **PAQ7 first as the understandable experimental baseline**.
- Validate your proposed mechanism on **at least one PAQ8 variant**.
- Present the contribution as a method for **PAQ-family/context-mixing compression**, rather than as a historical PAQ7 port.

Having both algorithms is valuable because demonstrating the same approach on both greatly strengthens the generality claim. But PAQ8 is a family of many variants, not one single stable algorithm, so state the exact version, source, options, and hash of the code.

Publication prospects

BSc thesis

Very suitable, even if the resulting speedup is moderate. The problem contains:

- algorithm analysis;
- CUDA architecture;
- dependency transformation;
- performance engineering;
- experimental methodology;
- compression-ratio/performance trade-offs.

Workshop, student conference, or regional conference

Realistic, provided that you produce a correct implementation, comprehensive comparison, and a measurable new trade-off.

Strong peer-reviewed compression/HPC paper

Possible but difficult. Since CuCM has now demonstrated GPU context mixing and 12.6× throughput improvement [79], you need an advance beyond the general idea. The strongest possibilities are:

- improved batch-update mathematics;
- a better PAQ-specific speed/ratio frontier;
- efficient exact multi-stream PAQ;
- a new speculative decoder;
- demonstrated generality across PAQ7 and PAQ8;
- public reproducible implementation and extensive evaluation.

Do not promise a significant paper before profiling. Publication depends on the measured result, not the implementation effort.

A realistic scope warning

A complete direct conversion of every PAQ7/PAQ8 model to CUDA is risky for a BSc timeline. PAQ contains:

- serial arithmetic coding;
- adaptive mutable state;
- irregular hash-table memory access;
- branch-heavy file-type models;
- small computations repeated at bit granularity;
- strong dependencies between prediction and update.

Start by profiling. Your source indicates likely targets such as the mixer's dot product and training operations, but their arithmetic intensity may be too low in isolation. The decisive question is whether sufficient independent work can be exposed without losing too much compression.

Bottom line

Proceed with the thesis, but revise the novelty claim.

A suitable claim is:

We design and evaluate a dependency-aware GPU parallelization strategy for PAQ-family context-mixing compressors, quantifying

and improving the trade-off between online adaptation, compression effectiveness, and throughput.

An unsuitable claim is:

This is the first CUDA context-mixing or PAQ-like compressor.

The newly published CuCM work makes that claim untenable [79]. Your best route is **block/vector parallelism plus a new state warm-up or delayed-update technique**, tested first on PAQ7 and then validated on PAQ8.